

Amazon Web Services

Complete Learning Guide

Beginner-Friendly · In-Depth · Practical

A comprehensive reference covering all major AWS Developer Tools — from command-line access and infrastructure-as-code to fully managed CI/CD pipelines — with real-world use cases, architecture diagrams, comparison tables, and hands-on code examples.

Services Covered

#	Service	Core Purpose
1	AWS CLI	Command-Line Interface — Control All of AWS from Your Terminal
2	AWS CDK	Cloud Development Kit — Infrastructure as Code in Your Language
3	AWS CodeBuild	Fully Managed Build Service — Compile, Test & Package
4	AWS CodeCommit	Managed Git Repositories — Secure Private Source Control
5	AWS CodeDeploy	Automated Deployment — Any Instance, Any Platform
6	AWS CodePipeline	Fully Managed CI/CD — Automate Your Release Pipeline

Version 1.0 · AWS Certified Training Material

Table of Contents

1. AWS CLI — *Installation · Configuration · Commands · Profiles · Output Formats*

- 1.1 Introduction & Analogy
- 1.2 Core Concepts
- 1.3 Architecture & Working
- 1.4 Features
- 1.5 Real-World Use Cases
- 1.6 Practical Example
- 1.7 Integrations & Comparison

2. AWS CDK — *Constructs · Apps · Stacks · Synth · Deploy · L1/L2/L3*

- 2.1–2.7 Full Coverage

3. AWS CodeBuild — *Build Projects · Buildspec · Phases · Artifacts · Caching*

- 3.1–3.7 Full Coverage

4. AWS CodeCommit — *Repositories · Branches · PRs · Triggers · Approval Rules*

- 4.1–4.7 Full Coverage

5. AWS CodeDeploy — *Applications · Deployment Groups · AppSpec · EC2 · Lambda · ECS*

- 5.1–5.7 Full Coverage

6. AWS CodePipeline — *Stages · Actions · Artifacts · Approvals · Multi-Region*

- 6.1–6.7 Full Coverage

Appendix: Master Comparison Table — *All 6 Services Side-by-Side + CI/CD Architecture*

AWS Command Line Interface (CLI)

Control Every AWS Service from Your Terminal — Automate, Script & Explore

1.1 Simple Introduction

What is the AWS CLI?

The AWS Command Line Interface (CLI) is a unified open-source tool that allows you to interact with AWS services directly from your terminal or command prompt using simple text commands. Instead of clicking through the AWS Management Console, you type structured commands like **aws s3 cp** or **aws ec2 describe-instances**. Every action available in the Console is also available via the CLI — and many advanced operations are CLI-only. The CLI is the foundation for scripting, automation, and CI/CD pipelines.

Why Does It Exist?

The AWS Console is great for exploration and one-off tasks, but it is not scriptable, not reproducible, and extremely slow for bulk operations. If you need to copy 10,000 files to S3, start 50 EC2 instances, or list all unencrypted EBS volumes across 10 regions — the Console would take hours of manual clicks. The CLI does this in a single command or a short script.

■ **The AWS CLI is like the keyboard shortcut system for AWS.** Using the Console is like clicking through menus with a mouse — intuitive but slow. The CLI is like knowing all the keyboard shortcuts — faster, repeatable, and chainable. A DevOps engineer who knows the CLI can do in 10 seconds what takes a junior admin 10 minutes in the Console. And unlike the Console, CLI commands can be saved in scripts, version-controlled, and run automatically on a schedule.

Business Problems it Solves

Problem	How CLI Solves It
Manual Console clicks for repetitive tasks	Write a bash/Python script using CLI — run once, run forever
Onboarding 50 new IAM users manually	Loop in shell script: <code>aws iam create-user</code> for each user in a CSV
Syncing local files to S3 on every deploy	<code>aws s3 sync ./dist s3://my-bucket</code> — one command, handles deltas
Auditing all EC2 instances across 15 regions	Shell loop + <code>aws ec2 describe-instances --region</code> — aggregate and filter with <code>jq</code>
CI/CD pipelines need to deploy AWS resources	CLI commands in pipeline scripts — no Console interaction needed
Testing CloudFormation changes quickly	<code>aws cloudformation deploy</code> — deploy stack without Console upload steps

1.2 Core Concepts & Terminology

Term	Definition
AWS CLI v2	Current major version (v2). Improved installer, AWS SSO support, auto-prompt, better binary transfer. Recommended for all new usage.
Command Structure	<code>aws [--parameter value ...] — e.g., <code>aws s3 ls s3://my-bucket --recursive</code></code>
Named Profile	A named set of credentials and config (access key, region, output format). Switch with <code>--profile</code> . Default profile used when no <code>--profile</code> specified.
Configuration File	<code>~/.aws/config</code> — stores profiles, regions, output formats, SSO settings.
Credentials File	<code>~/.aws/credentials</code> — stores access key ID and secret access key per profile. Never commit to Git.
Region	Specified via <code>--region</code> flag, <code>AWS_DEFAULT_REGION</code> env var, or config file. Determines which AWS region the command targets.
Output Format	How results are displayed: json (default), yaml, text (tab-delimited), table (ASCII table). Set per command or in config.
<code>--query</code>	JMESPath expression to filter/transform CLI JSON output client-side before display. Extremely powerful for extracting specific fields.
<code>--dry-run</code>	Available on some commands (e.g., <code>ec2 run-instances</code>) — validates permissions and parameters without actually executing.
<code>AWS_PROFILE</code>	Environment variable to set the active profile without <code>--profile</code> flag — useful in scripts and CI/CD.
Pagination	For commands returning many items, CLI paginates automatically. Use <code>--page-size</code> and <code>--max-items</code> to control. <code>--no-paginate</code> to disable.
Wait Commands	<code>aws ec2 wait instance-running --</code> polls until EC2 instance is running. Blocks script until condition is met.
AWS SSO (IAM Identity Center)	CLI v2 supports SSO login: <code>aws sso login --profile my-sso-profile</code> . No long-term access keys needed.
Environment Variables	<code>AWS_ACCESS_KEY_ID</code> , <code>AWS_SECRET_ACCESS_KEY</code> , <code>AWS_SESSION_TOKEN</code> , <code>AWS_DEFAULT_REGION</code> — override config file settings.

1.3 Architecture & Working

How a CLI Command Executes

USER types: `aws s3 ls s3://my-bucket`

↓

CLI CLIENT (Python-based, installed locally)

1. Parse command: `service=s3`, `action=ls`, `param=s3://my-bucket`
2. Load credentials: `~/.aws/credentials` or env vars or IAM role (if on EC2)
3. Load config: region, output format from `~/.aws/config` or `--flags`
4. Build HTTP request: `GET https://s3.amazonaws.com/my-bucket?list-type=2`
5. Sign request with AWS Signature Version 4 (SigV4) using credentials

↓ HTTPS request over internet (or VPC endpoint)

AWS SERVICE API (S3 in this case)

Validates SigV4 signature → checks IAM permissions → executes action

Returns XML/JSON response

↓

CLI CLIENT (processes response)

Apply `--query JMESPath` filter (if specified)

Format output: `json / yaml / text / table`

Print to stdout → user sees result

Credential Provider Chain (Priority Order)

Priority	Source	When Used
1 (highest)	Command-line <code>--profile</code> flag	Explicit override for this command only
2	Environment variables (<code>AWS_ACCESS_KEY_ID</code>)	CI/CD pipelines, Docker containers
3	AWS SSO credentials (<code>~/.aws/sso/cache</code>)	Developer machines using AWS SSO login
4	<code>~/.aws/credentials</code> file	Developer machines with named profiles
5	<code>~/.aws/config</code> file	Assume-role profiles defined in config
6	Container credentials (ECS task role)	Running inside ECS tasks/Fargate
7 (lowest)	EC2 Instance Metadata Service (IMDSv2)	Running on EC2 — uses attached IAM role

1.4 Major Features

Feature	Description	Example
Auto-Prompt	Interactive command builder — press F5 or use <code>--cli-auto-prompt</code> for guided command construction with docs inline.	<code>aws --cli-auto-prompt</code>
<code>--query</code> (JMESPath)	Filter and transform JSON output. Extract specific fields, list IDs, sort results.	<code>aws ec2 describe-instances --query 'Reservations[*].Instances[*].InstanceId' --output text</code>
Named Profiles	Multiple AWS accounts/roles in one config. Switch per command or via <code>AWS_PROFILE</code> .	<code>aws s3 ls --profile production</code>
AWS SSO Login	Browser-based login via AWS IAM Identity Center — no static access keys.	<code>aws sso login --profile dev-account</code>
S3 High-Level (s3)	Intuitive s3 commands: <code>cp</code> , <code>mv</code> , <code>sync</code> , <code>ls</code> , <code>rm</code> with recursive and progress support.	<code>aws s3 sync . s3://bucket --delete</code>
Wait Subcommands	Block script execution until a resource reaches a desired state.	<code>aws ec2 wait instance-running --instance-ids i-123</code>
<code>--dry-run</code>	Test if you have permissions for an action without executing it.	<code>aws ec2 run-instances --dry-run ...</code>

[illegible]

Service / Tool	How CLI Relates
AWS Console	CLI is the scriptable alternative to every Console action. Same underlying APIs.

Service / Tool	How CLI Relates
AWS SDKs	SDKs (boto3, JS SDK) used in application code; CLI used for ops/admin scripts and terminals.
AWS CloudFormation	aws cloudformation deploy / create-stack / describe-stack-events — deploy IaC from scripts.
AWS CDK	CDK uses CLI under the hood for cdk deploy. CDK bootstrap calls aws cloudformation.
CI/CD Pipelines	CLI commands in CodeBuild buildspec, GitHub Actions, GitLab CI — deploy, test, notify.
AWS Systems Manager	aws ssm start-session — open Session Manager shell without SSH, via CLI.
AWS Secrets Manager	aws secretsmanager get-secret-value — retrieve secrets in scripts without hard-coding.

Dimension	AWS CLI	AWS SDK (e.g., boto3)	AWS Console
Interface	Terminal / shell	Application code (Python, JS, Java)	Web browser
Use Case	Ops scripts, CI/CD	Application logic, automation	Exploration, one-off tasks
Scriptable	Yes	Yes	No
Auth	Config file / SSO	Same credential chain as CLI	IAM username or SSO
Best For	DevOps, sys admins	Developers building apps	Beginners, visual exploration

AWS Cloud Development Kit (CDK)

Define Cloud Infrastructure in Python, TypeScript, Java & More — Then Synthesize to CloudFormation

2.1 Simple Introduction

What is the AWS CDK?

The AWS Cloud Development Kit (CDK) is an open-source software development framework that lets you define your entire AWS cloud infrastructure using familiar programming languages — Python, TypeScript, JavaScript, Java, C#, and Go. Instead of writing hundreds of lines of YAML/JSON CloudFormation templates, you write concise, object-oriented code using the CDK library. The CDK then **synthesizes** your code into a CloudFormation template and **deploys** it using CloudFormation. The result is infrastructure defined with the same tools, IDE support, and software engineering practices (classes, loops, conditionals, tests) as application code.

The Core Insight

CloudFormation YAML is declarative but verbose and limited — no loops, no functions, no abstractions, no type safety. CDK brings the full power of a programming language to infrastructure: create a VPC with a loop, define a reusable security pattern as a class, test your infrastructure logic with pytest/jest, use autocomplete in your IDE.

■ **CDK is like LEGO — but you program the bricks.** Raw CloudFormation is like buying every LEGO piece individually and placing each one by hand with precise coordinates. CDK gives you pre-designed LEGO sets (Constructs) — a VPC set, an ECS Fargate set, an API Gateway + Lambda set. You snap them together in code, add your customizations, and CDK assembles the final model (CloudFormation template). Advanced users can even create their own custom LEGO pieces (custom Constructs) and share them with the team.

2.2 Core Concepts & Terminology

Term	Definition
App	The root of a CDK application. Contains one or more Stacks. Entry point of every CDK program.
Stack	A deployable unit in CDK — maps 1:1 to a CloudFormation stack. Contains Constructs (resources).
Construct	The basic building block of CDK. Represents one or more AWS resources. Three levels: L1, L2, L3.
L1 Construct (Cfn*)	Low-level — direct mapping to a CloudFormation resource. e.g., CfnBucket. Full control, verbose. Auto-generated from CloudFormation spec.
L2 Construct	High-level, opinionated — wraps L1 with sensible defaults and helper methods. e.g., s3.Bucket, lambda_.Function, ec2.Vpc. Most commonly used.
L3 Construct (Patterns)	Highest level — combines multiple L2 constructs into a complete pattern. e.g., aws_ecs_patterns.ApplicationLoadBalancedFargateService (creates ALB + ECS + IAM in one line).
cdk synth	Synthesize: converts your CDK code into a CloudFormation template (YAML/JSON). No deployment happens.

Term	Definition
cdk deploy	Deploy: synthesizes the template and calls CloudFormation to create/update the stack.
cdk diff	Shows what infrastructure changes will occur if you deploy — like a CloudFormation Change Set preview.
cdk destroy	Deletes the CloudFormation stack and all its resources.
cdk bootstrap	Creates a CDKToolkit CloudFormation stack in your account/region — S3 bucket and IAM roles needed for CDK to operate.
CDK Context	Key-value store for runtime configuration — environment-specific values (account IDs, VPC IDs) stored in cdk.context.json.
Aspects	Visitor pattern — apply a function to every Construct in the app tree. Used for enforcing org-wide policies (e.g., tag all resources).
CDK Construct Hub	Public registry (constructs.dev) of open-source CDK constructs from AWS and the community.
CDK Pipelines	L3 pattern for self-mutating CI/CD pipelines using CodePipeline — the CDK pipeline updates itself when you commit CDK code changes.

2.3 Architecture & Working

DEVELOPER writes CDK app in Python/TypeScript/Java/C#/Go
 Uses IDE: autocomplete, type checking, inline docs for all CDK constructs
 ↓ cdk synth
CDK FRAMEWORK (runs locally – open source npm package)
 Instantiates App → Stacks → Constructs
 Resolves tokens, asset references, cross-stack dependencies
 Generates: CloudFormation template JSON + cdk.out/ directory
 ↓ cdk deploy
CDK CLI
 Uploads assets (Lambda code, Docker images) to CDK bootstrap S3/ECR
 Calls CloudFormation CreateChangeSet / ExecuteChangeSet
 ↓
AWS CLOUDFORMATION (provisions all resources)
 Creates/updates/deletes resources in dependency order
 Rolls back automatically on failure
 ↓
DEPLOYED INFRASTRUCTURE (EC2, Lambda, VPC, RDS, S3, IAM...)

L1 vs L2 vs L3 Construct Comparison

Level	Name	Example	Lines of Code	Control vs Convenience
L1	CloudFormation Resource	CfnBucket(self, 'B', bucket_name='my-bucket')	~Same as YAML	Full control, verbose
L2	AWS Construct Library	s3.Bucket(self, 'B', versioned=True, encryption=...)	5–20 lines	Balanced — opinionated defaults

Level	Name	Example	Lines of Code	Control vs Convenience
L3	Patterns / Solutions	<code>ecs_patterns.ApplicationLoadBalancedFargateService(self,...)</code>	2–5 lines	Maximum convenience, less control

2.4 Major Features

Feature	Description
Multi-Language Support	Python, TypeScript, JavaScript, Java, C#, Go — use your team's existing language expertise.
IDE Integration	Full autocomplete, type safety, inline documentation for all 300+ CDK constructs in any IDE.
Construct Library	300+ L2 constructs covering every major AWS service with sensible secure defaults.
Aspects (Policy as Code)	Apply visitor functions to enforce org standards: tag every resource, enforce encryption, check compliance.
CDK Pipelines	Self-mutating CI/CD pipeline using CodePipeline. Pipeline updates itself when you change CDK code.
Asset Handling	Lambda code, Docker images, local files — CDK packages, uploads to S3/ECR, and references them in CloudFormation automatically.
Environment Agnostic Stacks	Write once, deploy to any account/region using CDK environments or env-agnostic tokens.
Testing (Fine-Grained Assertions)	Use pytest/jest to test your CDK code: assert a bucket has encryption, assert a Lambda has the right timeout.
Construct Hub	Public registry of open-source CDK constructs — reuse community patterns (e.g., projen, datadog-cdk-constructs).
cdk watch	Hot-reload mode — monitors source files and auto-deploys changes (great for Lambda development).

2.5 Real-World Use Cases

Scenario	CDK Advantage
Multi-env deployments	One CDK app: deploy to dev/staging/prod with different params (instance size, replication) using Python dictionaries.
Serverless API (1 pattern)	<code>api_gateway.LambdaRestApi</code> or <code>apigwv2_integrations.HttpLambdaIntegration</code> — full API + Lambda in 3 lines.
Enforce org tagging policy	<code>cdk.Aspects</code> : apply a custom Aspect that tags every resource with team, cost-center, environment — guaranteed, no exceptions.
ECS Fargate service	<code>ecs_patterns.ApplicationLoadBalancedFargateService</code> — ALB + ECS cluster + task + IAM + auto-scaling in 10 lines.
VPC with custom CIDRs	<code>ec2.Vpc</code> with <code>subnet_configuration</code> — define public/private/isolated subnets, NAT gateways, route tables in one object.

[illegible]

2.7 Integrations & Comparison

Service	Integration		
AWS CloudFormation	CDK synthesizes to CloudFormation. All deployments use CloudFormation under the hood.		
AWS CodePipeline	CDK Pipelines creates a self-mutating CodePipeline from CDK code automatically.		
AWS CodeBuild	CDK Pipelines uses CodeBuild to run cdk synth and cdk deploy stages in the pipeline.		
AWS CLI	CDK CLI uses AWS CLI's credential chain. CDK deploy calls CloudFormation APIs via CLI.		
Construct Hub	constructs.dev — public registry of open-source CDK constructs from AWS, community, partners.		
Dimension	AWS CDK	AWS CloudFormation (raw)	Terraform
Language	Python, TS, Java, C#, Go	YAML / JSON	HCL

Dimension	AWS CDK	AWS CloudFormation (raw)	Terraform
Abstraction	High (L2/L3 constructs)	Low (every property)	Medium
Loops/Conditions	Yes (native language)	Limited (Conditions/Map)	Yes (count, for_each)
IDE Support	Excellent (typed autocomplete)	Basic	Good (HCL plugins)
Testing	Yes (pytest, jest assertions)	cfn-lint only	Terratest
Output	CloudFormation template	CloudFormation template	Terraform plan/apply
State Management	CloudFormation (no state file)	CloudFormation	state file required
Best For	Developers preferring code	Ops teams comfortable with YAML	Multi-cloud teams

AWS CodeBuild

Fully Managed Build Service — Compile Code, Run Tests & Produce Artifacts

3.1 Simple Introduction

What is AWS CodeBuild?

AWS CodeBuild is a fully managed continuous integration service that compiles your source code, runs unit tests, and produces deployment-ready build artifacts — all without you provisioning or managing any build servers. You define your build steps in a **buildspec.yml** file, CodeBuild spins up a fresh Docker container, executes your steps, produces artifacts (JARs, ZIPs, Docker images), and then terminates the container. You pay only for the compute time used during the build.

Why Does It Exist?

Traditional CI requires maintaining Jenkins servers, build agents, plugins, and managing scaling during peak load (e.g., 50 developers pushing code simultaneously). CodeBuild is completely serverless — it auto-scales to run thousands of parallel builds with no infrastructure to manage. Each build runs in a fresh, isolated container — no 'works on my build server' problems.

■ **CodeBuild is like a self-cleaning factory with unlimited production lines.** Each time an order arrives (build triggered), a brand new production line spins up (Docker container), assembles the product (compile + test), packages it (artifact), and then the entire production line is cleaned up and disappears. If 1,000 orders arrive simultaneously, 1,000 parallel production lines spin up. You never maintain the factory — AWS does.

3.2 Core Concepts & Terminology

Term	Definition
Build Project	The configuration object that defines: source, environment (Docker image), compute size, buildspec, artifacts, cache, and VPC settings.
buildspec.yml	YAML file in your repository root that defines build phases, environment variables, and artifact paths. The heart of CodeBuild.
Build Phase	A stage in the buildspec: <code>install</code> → <code>pre_build</code> → <code>build</code> → <code>post_build</code> . Each phase has commands. Failure in any phase (except <code>post_build</code>) stops the build.
Environment Image	Docker image used as the build environment. AWS provides curated images (Amazon Linux 2, Ubuntu, Windows Server). You can also specify any custom Docker image from ECR or Docker Hub.
Compute Type	Size of the build container: <code>BUILD_GENERAL1_SMALL</code> (3 GB RAM / 2 vCPU) to <code>BUILD_GENERAL1_2XLARGE</code> (145 GB / 72 vCPU). Choose based on build requirements.
Source	Where CodeBuild fetches source code: CodeCommit, GitHub, GitLab, Bitbucket, S3, or no source (build from a Docker image).
Artifact	Output of the build: files uploaded to S3. Defined by files pattern in buildspec. Can be encrypted, versioned.

Term	Definition
Secondary Sources	Additional source repositories fetched alongside the primary source — for monorepos or shared libraries.
Cache	Store downloaded dependencies (Maven, npm, pip packages) in S3 or locally between builds. Dramatically reduces build times.
Environment Variable	Key-value pairs available during the build. Three types: Plaintext, Parameter Store (SSM), Secrets Manager.
Build Badge	Dynamic badge (SVG URL) showing latest build status — embed in your README.
Report Group	Collect and visualize unit test results, coverage reports from builds in CodeBuild console over time.
VPC Configuration	Run builds inside your VPC to access private resources (RDS, internal APIs) during build/test phases.
Privileged Mode	Required for builds that need to run Docker commands (docker build, docker push) inside the build container.

3.3 Architecture & Working

TRIGGER

CodePipeline → starts build ■ GitHub webhook → starts build ■ Manual / API

↓

CODEBUILD SERVICE

1. Provisions fresh Docker container (environment image + compute type)
2. Fetches source code from CodeCommit / GitHub / S3
3. Restores cache from S3 (if configured)

↓ Executes buildspec.yml phases:

INSTALL PHASE – install runtime versions, system packages

PRE_BUILD PHASE – login to ECR, set env vars, pre-checks

BUILD PHASE – compile code, run tests, docker build

POST_BUILD PHASE– docker push, notifications, cleanup

↓

ARTIFACTS → uploaded to S3 (or ECR for Docker images)

CACHE → saved to S3 for next build

LOGS → streamed to CloudWatch Logs + S3 in real-time

REPORTS → test results, coverage sent to Report Group

↓ Container terminated – no residual state

BUILD STATUS: SUCCEEDED / FAILED / STOPPED → notifies CodePipeline / SNS

3.4 Major Features

Feature	Description
Fully Managed / Serverless	No build servers to provision. AWS scales builds automatically — run 1 or 10,000 concurrent builds.

Feature	Description
Custom Build Environments	Use any Docker image (public or private ECR) as build environment — full control over tools.
Build Caching	Cache pip/npm/maven dependencies in S3 or local disk — reduce build time by 60–80%.
Test Reporting	JUnit/NUnit XML reports visualized in CodeBuild console with pass/fail trends over time.
VPC Builds	Connect build containers to your VPC for access to private databases, APIs, and services.
Docker Support	Build and push Docker images with privileged mode — full Docker daemon access inside builds.
Secrets Integration	Reference SSM Parameter Store and Secrets Manager values in buildspec — no plaintext secrets.
Batch Builds	Define multiple build configurations in one project — run them in parallel or sequence.
GitHub/GitLab Integration	Native webhooks: auto-trigger builds on push, PR open/update, tag creation.
Build Badges	Shareable SVG badge (passing/failing) for embedding in repository README files.

3.5 Real-World Use Cases

Scenario	Details
Python Lambda build	Install dependencies, run pytest, package Lambda ZIP, upload to S3 → CodePipeline picks up artifact for deployment.
Docker image CI	On every PR: docker build → run container-based tests → if passing: docker push to ECR → CodePipeline deploys to ECS.
Maven Java build	Cache ~/.m2/repository in S3. mvn clean package → run JUnit → upload JAR artifact → report results to Report Group.
React / Next.js frontend	npm ci → npm test → npm run build → aws s3 sync ./build s3://my-site → CloudFront invalidation.
Security scanning in CI	Pre-build: run trivy for Docker vulnerability scan, bandit for Python SAST, checkov for IaC — fail build on HIGH findings.
Integration test environment	VPC-connected build: spin up test database in RDS → run integration tests against it → tear down via CLI.

3.6 Practical Example — buildspec.yml for Python Lambda

```
# buildspec.yml – place in repository root
version: 0.2

env:
  variables:
    PYTHON_VERSION: "3.12"
    S3_ARTIFACT_BUCKET: "my-artifact-bucket"
parameter-store:
```

```

    SONAR_TOKEN: "/ci/sonar-token"      # fetched from SSM Parameter Store
secrets-manager:
    PYPI_TOKEN: "ci/pypi-token:token" # fetched from Secrets Manager

phases:
install:
    runtime-versions:
        python: "3.12"
    commands:
        - echo "Installing dependencies..."
        - pip install --upgrade pip
        - pip install -r requirements.txt
        - pip install pytest pytest-cov bandit

pre_build:
    commands:
        - echo "Running security scan..."
        - bandit -r src/ -ll          # fail on medium+ severity issues
        - echo "Build started on $(date)"

build:
    commands:
        - echo "Running tests with coverage..."
        - pytest tests/ -v
            --junitxml=reports/test-results.xml
            --cov=src
            --cov-report=xml:reports/coverage.xml
        - echo "Packaging Lambda function..."
        - mkdir -p dist
        - pip install -r requirements.txt -t dist/
        - cp -r src/* dist/
        - cd dist && zip -r ../lambda-package.zip . && cd ..

post_build:
    commands:
        - echo "Build completed: $CODEBUILD_BUILD_SUCCEEDING"
        - aws s3 cp lambda-package.zip s3://$S3_ARTIFACT_BUCKET/lambda/

artifacts:
files:
    - lambda-package.zip
name: LambdaPackage

reports:
pytest-report:
    files:
        - reports/test-results.xml
    file-format: JUNITXML
coverage-report:
    files:
        - reports/coverage.xml
    file-format: COBERTURAXML

cache:
paths:
    - /root/.cache/pip/**/*      # cache pip downloads between builds

```

3.7 Integrations & Comparison

Service	Integration
AWS CodePipeline	CodeBuild is the Build stage action in CodePipeline. Receives source artifact, returns build artifact.
Amazon S3	Source input and artifact output. Build cache also stored in S3.
Amazon ECR	Build Docker images and push to ECR. CodeBuild pre-auth with ecr get-login-password.
AWS CodeCommit	Native source provider — CodeBuild pulls source directly from CodeCommit repos.
CloudWatch Logs	All build logs streamed in real-time to CloudWatch Log Groups for monitoring and debugging.
AWS CDK	CDK Pipelines uses CodeBuild to run cdk synth. CodeBuild runs in a managed CodePipeline stage.
GitHub / GitLab	OAuth-connected source with webhook triggers — build on every push, PR, or tag.

Dimension	AWS CodeBuild	Jenkins	GitHub Actions
Infrastructure	Fully managed (serverless)	Self-managed (EC2/K8s agents)	GitHub-managed runners
Scaling	Auto (unlimited parallel builds)	Manual (provision more agents)	Auto (limited by plan)
AWS Integration	Native (IAM, SSM, ECR, S3)	Via plugins	Via aws-actions
Cost	Per build-minute	EC2 + licensing cost	Free tier + per-minute
Environment	Any Docker image	Any Docker / bare metal	GitHub-hosted or self-hosted
Best For	AWS-native CI/CD	Complex pipelines, self-hosted	Open-source / GitHub repos

AWS CodeCommit

Fully Managed Private Git Repositories — Secure, Scalable Source Control on AWS

4.1 Simple Introduction

What is AWS CodeCommit?

AWS CodeCommit is a fully managed source control service that hosts secure and highly scalable private Git repositories. CodeCommit uses standard Git protocols — git clone, git push, git pull all work identically to GitHub or GitLab. Your repositories are encrypted at rest using AWS KMS, encrypted in transit using HTTPS/SSH, and access-controlled using AWS IAM. There are no repository size limits and no restrictions on file types. Since it's fully managed by AWS, you never provision servers, manage storage, or worry about availability.

Why Use CodeCommit?

For teams that need their source code to stay entirely within AWS infrastructure for compliance (HIPAA, FedRAMP, PCI-DSS, ITAR), or want to use IAM policies for code access control without managing a separate identity system, or need native integration with AWS CodePipeline, CodeBuild, and CodeDeploy without OAuth tokens or webhooks — CodeCommit provides all of this natively.

■ *Note: AWS announced in July 2024 that CodeCommit will not accept new customers. Existing customers can continue using it, but AWS recommends evaluating migration to alternatives like GitHub, GitLab, or Bitbucket for new projects. It remains important for the AWS Certified Developer exam and existing AWS pipelines.*

■ **CodeCommit is like a private, fireproof library vault inside AWS.** GitHub/GitLab are public libraries (with private rooms) — your code is stored on their infrastructure. CodeCommit is a vault inside your own AWS account — the same IAM badge that lets you into EC2 and S3 lets you into CodeCommit. No separate login system, no third-party infrastructure, no compliance exceptions to write.

4.2 Core Concepts & Terminology

Term	Definition
Repository	A Git repository hosted in CodeCommit. Encrypted at rest (KMS), no size limit. Accessed via HTTPS or SSH.
Branch	Named pointer to a commit — identical to Git branches. Default branch configurable (main, master, etc.).
Commit	A snapshot of the repository at a point in time. Standard Git commit with author, timestamp, message, SHA.
Pull Request (PR)	A request to merge changes from one branch to another. Supports: description, reviewers, comments, approval rules, diff view.
Approval Rule Template	An organization-wide or repository rule that requires N approvals from specific IAM users/groups before a PR can be merged.
Trigger	Event-driven notification: on push, branch create, or PR — invoke Lambda or send SNS notification.
Notification Rule	Send CodeCommit events (PR created, comment, branch push) to SNS topic → Slack, email, PagerDuty.

Term	Definition
Repository Tag	AWS resource tags on the repository for cost allocation and access control (ABAC with IAM tag conditions).
HTTPS (GRC)	Git Remote CodeCommit — helper that uses AWS credentials for HTTPS authentication without IAM HTTPS Git credentials.
SSH Key	Upload your SSH public key to your IAM user → use standard SSH git URLs for authentication.
Diff / Comparison	Visual file-by-file diff between any two branches, commits, or tags — in the Console or API.

4.3 Architecture & Working

DEVELOPER

```
git clone https://git-codecommit.ap-south-1.amazonaws.com/v1/repos/my-repo
(Auth: IAM HTTPS credentials OR AWS CLI credential helper OR SSH key)
```

```
↓ git push origin feature/my-feature
```

CODECOMMIT SERVICE (fully managed, multi-AZ, AWS infrastructure)

Authenticates request against IAM policies

Validates IAM permission: codecommit:GitPush on the repository resource

Stores commits encrypted with KMS key

Replicates across multiple AZs for durability

```
↓ Push accepted
```

TRIGGERS & NOTIFICATIONS (optional, event-driven)

Branch push → Lambda trigger (run linting / validation)

Push to main → SNS → CodePipeline source poll detects change → starts pipeline

PR created → SNS → Slack webhook notifies team

```
↓
```

CODEPIPELINE SOURCE STAGE

CodePipeline polls or receives EventBridge event from CodeCommit

Downloads source ZIP → passes to CodeBuild for CI

4.4 Major Features

Feature	Description
Standard Git	100% Git-compatible. Every Git client, IDE plugin, and tool works with CodeCommit unchanged.
IAM-Based Access Control	No separate username/password system. IAM policies control who can push, pull, create branches per repository.
Encryption	At-rest: KMS-encrypted (customer-managed or AWS-managed key). In-transit: TLS 1.2 required.
Pull Requests	Full PR workflow: branch comparison, line comments, reviewer assignment, approval rules, merge via console or API.

[illegible]

4.7 Comparison: CodeCommit vs GitHub vs GitLab

Dimension	AWS CodeCommit	GitHub	GitLab (Self-Managed)
Hosting	AWS (inside your account)	GitHub Inc. (Microsoft)	Your infrastructure
Access Control	AWS IAM (native)	GitHub users/teams	GitLab users/groups
Encryption	KMS (customer-managed)	GitHub-managed	You manage
Compliance	Stays in AWS account	Data on GitHub servers	Data on your servers
Built-in CI/CD	Via CodePipeline	GitHub Actions (built-in)	GitLab CI/CD (built-in)
New Customers	No (announced July 2024)	Yes	Yes
Cost	\$1/user/month + \$0.06/GB	Free – \$21/user/month	Free – \$29/user/month
Best For	AWS-locked compliance environments	Open source, broad ecosystem	Full DevOps platform, self-hosted

AWS CodeDeploy

Automated Deployment to EC2, Lambda, ECS & On-Premises — Zero Downtime Releases

5.1 Simple Introduction

What is AWS CodeDeploy?

AWS CodeDeploy is a fully managed deployment service that automates the process of deploying application code to EC2 instances, Lambda functions, ECS services, and even on-premises servers. CodeDeploy handles the complex mechanics of deploying to many servers simultaneously — stopping the old version, deploying the new version, running health checks, and automatically rolling back if the deployment fails. It supports sophisticated deployment strategies like blue/green deployments and canary releases — all without manual coordination.

Why Does It Exist?

Manually deploying application updates to a fleet of 500 EC2 instances involves SSH-ing to each server, stopping the app, copying new files, restarting, verifying health — and if something goes wrong, repeating this process in reverse for rollback. CodeDeploy automates all of this: deploy to all instances in parallel (or in batches), monitor health, and if more than N% fail, roll back automatically.

■ **CodeDeploy is like a fleet of delivery trucks with GPS tracking and return policy.** When you ship new software (a delivery), CodeDeploy sends trucks to every server (delivery address) simultaneously. Each truck knows the delivery instructions (AppSpec.yml). If a delivery fails (health check fails), the truck automatically returns the previous package (rollback). You track all deliveries on a live map (CodeDeploy console) and get a report when everything is delivered.

5.2 Core Concepts & Terminology

Term	Definition
Application	A name that identifies the application to be deployed. Top-level CodeDeploy resource.
Deployment Group	A set of targets (EC2 instances, Lambda function, ECS service) to deploy to. Configured with deployment strategy, load balancer, alarms, and rollback settings.
Deployment	A specific execution: deploy revision X to deployment group Y. Has a unique ID, status, and event log.
Revision	The deployable artifact: application code + AppSpec.yml stored in S3 or GitHub.
AppSpec.yml	Deployment specification file: defines where to copy files (EC2) or traffic weights (Lambda/ECS) and which lifecycle hook scripts to run at each phase.
Lifecycle Hook	Scripts run at specific points during deployment: BeforeInstall, AfterInstall, ApplicationStart, ApplicationStop, ValidateService. Run on each instance.
Deployment Configuration	Controls rollout speed: AllAtOnce, HalfAtATime, OneAtATime — or custom percentage-based configs.
In-Place Deployment	(EC2 only) Stop old app → deploy new app → restart — on same instances. Faster but has downtime risk.

Term	Definition
Blue/Green Deployment	Provision NEW instances (green) → deploy → shift load balancer traffic → terminate old (blue). Zero downtime.
Canary Deployment	(Lambda/ECS) Route X% traffic to new version for Y minutes. If healthy, shift 100%. If not, rollback.
Linear Deployment	(Lambda/ECS) Shift traffic in equal increments every N minutes until 100%.
CodeDeploy Agent	Software installed on EC2 and on-premises instances. Communicates with CodeDeploy service to execute deployments.
Deployment Trigger	SNS topic notified on deployment events (started, succeeded, failed) — for Slack/PagerDuty alerts.
Rollback	Automatic or manual re-deployment of the previous successful revision when alarms fire or deployment fails.

5.3 Architecture & Working

TRIGGER (CodePipeline Deploy Stage ■ CLI ■ Console)

↓

CODEDEPLOY SERVICE

1. Identify deployment group targets (by EC2 tag, ASG, Lambda alias, ECS service)
2. Download revision (AppSpec + code) from S3 or GitHub
3. Send deployment instructions to CodeDeploy Agent on each instance

↓ Per-instance lifecycle (EC2 In-Place example)

ApplicationStop → run stop script (graceful shutdown)

DownloadBundle → agent downloads revision from S3

BeforeInstall → run pre-install script (backup, pre-checks)

Install → copy files to destination directory

AfterInstall → run post-install script (config, permissions)

ApplicationStart → run start script (systemctl start app)

ValidateService → run health check script (curl localhost/health)

↓

SUCCESS: all instances healthy → deployment complete

FAILURE: >N% fail → CloudWatch alarm fires → auto-rollback to previous revision

↓

LOAD BALANCER (for blue/green): traffic shifted only after health checks pass

Deployment Strategies

Strategy	Description	Downtime	Rollback Speed	Best For
In-Place AllAtOnce	Deploy to all instances simultaneously	Yes (brief)	Redeploy (~mins)	Dev/test environments
In-Place HalfAtATime	Deploy to 50% → then remaining 50%	Partial	Redeploy	Staging; tolerate partial downtime

Strategy	Description	Downtime	Rollback Speed	Best For
In-Place OneAtATime	Deploy to one instance at a time	Minimal	Slow	Production; maximum safety
Blue/Green (EC2)	New instances deployed → traffic shifted → old terminated	None	Instant (keep blue)	Production; zero downtime
Canary (Lambda/ECS)	X% traffic to new, rest to old for N mins → full shift	None	Instant	Lambda/ECS; gradual rollout
Linear (Lambda/ECS)	Shift traffic in equal steps every N minutes	None	Instant	Lambda/ECS; steady rollout

5.4 Major Features

Feature	Description
EC2 + On-Premises	Deploy to AWS EC2 instances and on-premises servers using the same CodeDeploy workflow — hybrid deployments.
Lambda Traffic Shifting	Canary/Linear/AllAtOnce traffic shifting between Lambda function versions with automatic CloudWatch alarm rollback.
ECS Blue/Green	Replace ECS task set: new tasks deployed → ALB/NLB listener rules shifted → old tasks terminated. Zero downtime.
Automatic Rollback	Configure CloudWatch Alarms — if alarm fires during deployment, CodeDeploy automatically rolls back.
Lifecycle Hooks	Run custom scripts at 7 points during deployment — validate, backup, notify, health check.
Deployment Triggers	SNS notifications on deployment events — integrate with Slack, PagerDuty, or monitoring systems.
Integration with ASGs	Blue/Green deployment: CodeDeploy creates a new ASG with new instances → shifts traffic → deletes old ASG.
Concurrent Deployments	Deploy to multiple deployment groups simultaneously — multi-region, multi-environment deployments.

5.5 Real-World Use Cases

Scenario	Strategy Used
Deploy Node.js API to 100 EC2 instances	In-Place HalfAtATime — deploy to 50 first, validate, then remaining 50
Zero-downtime Lambda update	Canary10Percent5Minutes — 10% traffic to new version for 5 min → if healthy → 100%
ECS microservice update	Blue/Green — new task set → test → ALB listener switches → old terminated
On-premises + EC2 hybrid deployment	Mixed deployment group: on-prem servers + EC2 — same AppSpec, CodeDeploy handles both

[illegible]

5.7 Integrations & Comparison

Service	Integration
AWS CodePipeline	CodeDeploy is the Deploy stage action in CodePipeline — receives artifact from CodeBuild and deploys.
Amazon EC2 / ASG	Deploy to tagged EC2 instances or Auto Scaling Groups. Blue/green creates new ASG automatically.
AWS Lambda	Deploy new function versions with traffic-shifting strategies. Rollback via CloudWatch alarms.
Amazon ECS	Blue/green deployments for ECS services. Integrates with ALB/NLB target group switching.
Amazon CloudWatch	Monitor deployment health. Alarm-triggered automatic rollback if metrics breach thresholds.
Amazon SNS	Deployment event notifications — push to Slack/PagerDuty on success/failure.
AWS CodeCommit	Use CodeCommit as source for revisions in GitHub-based deployment flows via CodePipeline.

AWS CodePipeline

Fully Managed CI/CD — Automate Your Entire Release Process from Code to Production

6.1 Simple Introduction

What is AWS CodePipeline?

AWS CodePipeline is a fully managed continuous delivery service that automates your software release process. It models the entire release workflow — from code commit to production deployment — as a series of **stages** (Source → Build → Test → Deploy). Each stage contains **actions** that integrate with AWS services or third-party tools. When you push code to your repository, CodePipeline automatically detects the change, triggers the pipeline, and progresses the code through every stage — building, testing, and deploying — without any manual intervention.

What is CI/CD?

Concept	Definition	CodePipeline Role
Continuous Integration (CI)	Automatically build and test every code commit	Source + Build stages
Continuous Delivery (CD)	Automatically deliver tested code to a staging env	Test + Deploy-to-Staging stages
Continuous Deployment (CD)	Automatically deploy to production on every commit	Deploy-to-Prod stage (no manual approval)

■ **CodePipeline is like an automated Formula 1 pit crew system.** When a driver (developer) comes in for a pit stop (code push), the entire pit crew (pipeline) springs into action simultaneously — tyres changed (build), car inspected (tests), fuel added (integration tests), driver briefed (manual approval) — all in a precisely choreographed sequence. Without automation, each pit stop would take 20 minutes instead of 2 seconds. CodePipeline is that coordination system.

6.2 Core Concepts & Terminology

Term	Definition
Pipeline	A workflow definition with ordered stages. Triggered automatically on source changes. Has a unique ARN and execution history.
Stage	A logical phase in the pipeline: Source, Build, Test, Deploy, Approval. Stages run sequentially. Actions within a stage can run in parallel.
Action	A single task within a stage. Has a provider (CodeBuild, CodeDeploy, Lambda, CloudFormation, S3, etc.) and an action type (Source, Build, Test, Deploy, Invoke, Approval).
Pipeline Execution	One run of the entire pipeline triggered by a source change or manual trigger. Has a unique ID and status.
Artifact	Data passed between stages — typically source code ZIP or build output ZIP. Stored in CodePipeline's S3 artifact bucket.

Term	Definition
Input Artifact	The artifact a stage action reads as input (e.g., Build stage reads source code ZIP).
Output Artifact	The artifact a stage action produces as output (e.g., Build stage outputs compiled ZIP).
Transition	The connection between stages. Can be disabled to pause the pipeline at a specific point.
Manual Approval	A pipeline action that pauses execution until a designated IAM user approves (or rejects). Used for production gating.
Source Polling	CodePipeline periodically polls CodeCommit/S3 for changes. EventBridge-based detection is faster and recommended.
Pipeline Variables	Key-value pairs passed between stages and actions at runtime. Access with <code>#{variables.MyVar}</code> syntax.
Trigger	What starts a pipeline run: code push to repo, S3 object change, EventBridge rule, or manual start.
Rollback	Trigger a CodeDeploy rollback from CodePipeline if a deployment fails.
V2 Pipeline Type	Newer pipeline type (2023+) with improved features: pipeline-level variables, trigger filtering by branch/file path.

6.3 Architecture & Working — Full CI/CD Pipeline

DEVELOPER: `git push origin main`

↓ EventBridge detects CodeCommit push

STAGE 1: SOURCE

Action: CodeCommit (or GitHub, S3)

Output: SourceArtifact (source code ZIP in S3)

↓

STAGE 2: BUILD

Action: CodeBuild (runs `buildspec.yml`)

- Install deps, run unit tests, compile code

Input: SourceArtifact

Output: BuildArtifact (compiled ZIP in S3)

↓

STAGE 3: TEST

Action A: CodeBuild (integration tests) – parallel with Action B

Action B: CodeBuild (security scan: SAST, dependency check)

Input: BuildArtifact

Output: TestReport (saved to S3)

↓

STAGE 4: DEPLOY-STAGING

Action: CodeDeploy → Deploy to staging ECS service

↓

STAGE 5: APPROVAL

Action: Manual Approval – QA lead reviews staging → Approve / Reject

On Approve: pipeline continues. On Reject: pipeline stops.

↓

STAGE 6: DEPLOY-PRODUCTION

Action: CodeDeploy → Blue/Green Deploy to production ECS

CloudWatch Alarm triggers auto-rollback if error rate spikes

↓

SNS NOTIFICATION: pipeline success/failure → Slack / Email

6.4 Major Features

Feature	Description
Fully Managed	No servers to provision. AWS manages pipeline orchestration, artifact storage, and execution history.
Source Integrations	CodeCommit, GitHub (v1/v2), GitLab, Bitbucket, Amazon S3, Amazon ECR as source providers.
Parallel Actions	Actions within a stage run in parallel — run unit tests + security scan simultaneously to save time.
Manual Approval Gates	Pause pipeline and require human approval before proceeding — with SNS email notification and approval URL.
Multi-Region Deployment	Deploy to multiple AWS regions in a single pipeline — built-in cross-region artifact replication.
Multi-Account Deploy	Deploy to resources in other AWS accounts using cross-account IAM roles.
Pipeline Variables	Define and pass variables between stages — environment name, version number, feature flags.
Trigger Filtering (V2)	Start pipeline only on changes to specific branches, tags, or file paths — avoid unnecessary runs.
EventBridge Integration	Pipeline state changes (started, succeeded, failed) published as EventBridge events for alerting.
Third-Party Integrations	Jenkins, Snyk, Anchore, Aqua Security, Synopsys, JFrog, and many more as action providers.

6.5 Real-World Use Cases

Scenario	Pipeline Design
Serverless Lambda API	CodeCommit → CodeBuild (test+package) → CloudFormation (deploy SAM/CDK) → Manual Approval → Prod CloudFormation
Containerized ECS Microservice	GitHub → CodeBuild (docker build+push to ECR) → CodeDeploy ECS Blue/Green → SNS success notification
React Frontend	CodeCommit → CodeBuild (npm test+build) → S3 sync (staging) → Manual Approval → S3 sync (prod) + CloudFront invalidation
CDK Infrastructure Pipeline	CDK Pipelines: self-mutating pipeline — cdk synth in CodeBuild → CloudFormation to staging → approval → CloudFormation to prod

Scenario	Pipeline Design
Multi-Region Deploy	CodeBuild → Deploy us-east-1 → Deploy eu-west-1 (parallel) → Deploy ap-south-1 (parallel) → all-green notification
Data Pipeline	S3 new file → CodePipeline → CodeBuild (validate schema) → Glue Job trigger → Athena query validation → SNS report

6.6 Practical Example — Pipeline via CloudFormation / CDK

```
# Using AWS CDK (Python) to define a complete CodePipeline
from aws_cdk import (
    Stack, SecretValue,
    aws_codepipeline as codepipeline,
    aws_codepipeline_actions as actions,
    aws_codebuild as codebuild,
    aws_codedeploy as codedeploy,
    aws_sns as sns,
    aws_iam as iam,
)
from constructs import Construct

class CICDPipelineStack(Stack):
    def __init__(self, scope: Construct, id: str, **kwargs):
        super().__init__(scope, id, **kwargs)

        # Artifacts
        source_artifact = codepipeline.Artifact("SourceArtifact")
        build_artifact = codepipeline.Artifact("BuildArtifact")

        # SNS topic for notifications
        alert_topic = sns.Topic(self, "PipelineAlerts",
                                display_name="CI/CD Pipeline Alerts")

        # CodeBuild project (build + test)
        build_project = codebuild.PipelineProject(self, "BuildProject",
            build_spec=codebuild.BuildSpec.from_object({
                "version": "0.2",
                "phases": {
                    "install": {"commands": ["pip install -r requirements.txt"]},
                    "build": {"commands": ["pytest tests/ -v", "zip -r app.zip src/"]},
                },
                "artifacts": {"files": ["app.zip"]},
            }),
            environment=codebuild.BuildEnvironment(
                build_image=codebuild.LinuxBuildImage.STANDARD_7_0),
        )

        # Full pipeline definition
        pipeline = codepipeline.Pipeline(self, "AppPipeline",
            pipeline_name="my-app-pipeline",
            pipeline_type=codepipeline.PipelineType.V2,
            stages=[
                # Stage 1: Source
                codepipeline.StageProps(
                    stage_name="Source",
                    actions=[actions.CodeCommitSourceAction(
                        action_name="CodeCommit_Source",
                        repository=codecommit.Repository.from_repository_name(
```

```

        self, "Repo", "my-backend-service"),
        branch="main",
        output=source_artifact,
    )]
),
# Stage 2: Build & Test
codepipeline.StageProps(
    stage_name="Build",
    actions=[actions.CodeBuildAction(
        action_name="Build_Test",
        project=build_project,
        input=source_artifact,
        outputs=[build_artifact],
    )]
),
# Stage 3: Manual Approval for Production
codepipeline.StageProps(
    stage_name="Approval",
    actions=[actions.ManualApprovalAction(
        action_name="Approve_Production",
        notification_topic=alert_topic,
        additional_information="Review staging before approving prod
deploy",
    )]
),
# Stage 4: Deploy to Production
codepipeline.StageProps(
    stage_name="Deploy",
    actions=[actions.CodeDeployEcsDeployAction(
        action_name="Deploy_Production",
        deployment_group=ecs_deploy_group, # CodeDeploy DG
        app_spec_template_input=build_artifact,
        task_definition_template_input=build_artifact,
    )]
),
]
)

```

6.7 Integrations & Comparison

Service	Integration
AWS CodeCommit / GitHub	Source stage — monitors for code changes and triggers the pipeline.
AWS CodeBuild	Build and Test stages — compiles code, runs tests, produces artifacts.
AWS CodeDeploy	Deploy stage — deploys artifacts to EC2, Lambda, ECS with rollback support.
AWS CloudFormation	Deploy stage — create/update CloudFormation stacks (IaC deployments, CDK).
Amazon ECR	Source stage — trigger pipeline on new Docker image push to ECR.
Amazon S3	Artifact store for all pipeline stages. Also a source and deploy target.
AWS Lambda	Invoke stage — run custom Lambda functions as pipeline steps for any custom logic.
Amazon EventBridge	Pipeline state events → trigger automated responses (Slack alerts, rollback, metrics).

Dimension	AWS CodePipeline	GitHub Actions	Jenkins
Type	Managed CI/CD service	CI/CD in GitHub	Self-hosted CI/CD
Infrastructure	Fully managed by AWS	GitHub-managed runners	Self-managed servers
AWS Integration	Native (deep)	Via aws-actions (good)	Via plugins (moderate)
Configuration	Console / CloudFormation / CDK	YAML workflows in repo	Groovy Jenkinsfile
Parallel Actions	Yes (within a stage)	Yes (job matrix)	Yes (parallel stages)
Cost	\$1/active pipeline/month + actions	Free tier + per-min	EC2 cost + maintenance
Multi-Region	Built-in (cross-region actions)	Manual config	Manual config
Best For	AWS-native CI/CD with deep service integration	Open source, GitHub repos	Complex custom pipelines, self-hosted

Master Comparison: All 6 AWS Developer Tools

At-a-Glance Summary

Service	Type	Core Strength	Cost Model	Best For
AWS CLI	CLI Tool	Scriptable API access to all AWS services	Free (pay for API calls)	Ops scripts, automation, CI/CD
AWS CDK	IaC Framework	Infrastructure in Python/TypeScript/Java	Free (CloudFormation cost for deploy)	Developers who prefer code over YAML
AWS CodeBuild	Managed CI	Serverless build, test, package	Per build-minute	AWS-native CI, Docker builds
AWS CodeCommit	Source Control	IAM-integrated private Git (existing users)	\$1/user/month + storage	AWS-compliance environments
AWS CodeDeploy	Deployment	Blue/green, canary, rollback automation	Per deployment (EC2 free)	EC2/Lambda/ECS deployments
AWS CodePipeline	CI/CD Orchestrator	End-to-end pipeline orchestration	\$1/active pipeline/month	Full AWS-native CI/CD pipeline

Complete AWS CI/CD Pipeline Architecture

```
DEVELOPER: git push origin feature/new-api
↓
SOURCE (CodeCommit / GitHub)
EventBridge detects push to main / PR merge → triggers CodePipeline
↓
BUILD & TEST (CodeBuild)
Parallel actions:
Action A: Unit tests + code coverage (pytest) → Report Group
Action B: Security scan (bandit, trivy, checkov)
Action C: CDK synth → validate CloudFormation templates
Output: BuildArtifact (ZIP) + Docker image pushed to ECR
↓
DEPLOY TO STAGING (CodeDeploy / CloudFormation / CDK)
Blue/Green deploy to staging ECS service
Smoke tests run via CodeBuild action
↓
MANUAL APPROVAL (CodePipeline Approval Action)
SNS → email QA lead with staging URL and diff summary
QA approves in Console / Slack / CLI
↓
DEPLOY TO PRODUCTION (CodeDeploy)
Canary: 10% traffic to new version for 5 minutes
```

CloudWatch Alarm: if `ErrorRate > 1%` → auto-rollback
If healthy: 100% traffic shifted to new version
↓
NOTIFY (EventBridge → SNS → Slack)
'Pipeline succeeded: v2.3.1 deployed to production at 14:32 UTC'

When to Use Which Developer Tool

Q: I want to manage AWS from the terminal / write deployment scripts?

→ **AWS CLI**

Q: I want to define infrastructure in Python/TypeScript instead of YAML?

→ **AWS CDK**

Q: I need to automatically compile, test and package code on every push?

→ **AWS CodeBuild**

Q: I need a private Git repository that uses IAM for access control?

→ **AWS CodeCommit** (existing) or GitHub/GitLab (new projects)

Q: I need to deploy code to EC2 instances with zero downtime?

→ **AWS CodeDeploy** (blue/green deployment)

Q: I need to automate the entire flow from commit to production?

→ **AWS CodePipeline** (orchestrates CLI + CDK + CodeBuild + CodeDeploy)

Q: I need a fully automated, self-mutating infrastructure pipeline?

→ **AWS CDK Pipelines** (CodePipeline + CodeBuild + CDK synth + CloudFormation)

End of AWS Developer Tools Complete Guide

AWS CLI · AWS CDK · AWS CodeBuild · AWS CodeCommit · AWS CodeDeploy · AWS CodePipeline